

Guide to Module 2: Computational Spectroscopy & Signal Filtering

Llinersy Uranga Piña, LPPI - Cergy-Paris Université

June 2026

1 Introduction

Welcome to your data preprocessing template! In physical chemistry and materials science, raw instrument readouts are rarely publication-ready. Instrumental drifts, optical scattering, electronic thermal noise, and transient hardware glitches degrade our spectroscopic signals.

This notebook treats programming as a digital laboratory benchtop. Every function is an analytical instrument knob you can adjust to refine your raw measurements. This manual bridges the gap between physical observations and python parameters.

2 The Core Preprocessing Pipeline Explained

The script acts like a workflow chain. Altering a variable early in the pipeline impacts the downstream results.

2.1 1. Handling Missing Data: Interpolation

When spectrometers drop frames or detector pixels briefly saturate, they leave behind blanks or NaN (Not a Number) missing data indices.

- **The Knob:** `df['Absorbance'].interpolate(method='linear')`
- **Experimental Analogy:** Imagine drawing a straight, connecting pencil line on a paper graph to bridge a tiny tear in the page.
- **When to be cautious:** Linear interpolation works beautifully for micro-gaps (1–5 skipped data steps). However, if your experimental gap spans an important chemical band (e.g., your whole Carbonyl region), a straight line creates a dangerous artifact by smoothing away the real peak entirely.

2.2 2. Noise Reduction: Savitzky-Golay (SG) Filtering

Electronic white noise manifests as rapid, jagged ripples across your spectrum. Unlike aggressive moving-average blurs that crush peak heights, the Savitzky-Golay algorithm uses local calculus matrix fittings to separate noise from genuine molecular transitions.

- **The Knobs:** `window_size` (must be an odd integer) and `poly_order`.
- **How it operates:** The filter slides a small tracking window over your data. Inside that window, it fits a polynomial of choice (e.g., quadratic or cubic), evaluates the center point, and steps forward.

- **The Smoothing Trade-off:**

Small Window (e.g., 11) → Preserves sharp peaks, leaves high noise remnant.

Optimal Window (e.g., 31–51) → Maximizes signal-to-noise ratio (SNR).

Massive Window (e.g., 301) → Over-smoothes. Crushes peak height and artificially broadens width.

2.3 3. Removing Background Artifacts: Baseline Subtraction

Due to sample scattering, optical window fouling, or changing source temperatures, spectra typically float on a broad curve rather than resting at zero absorbance.

- **The Knob:** `np.polyfit(x, y, order)`

- **The Danger of Overfitting:** We use a low order polynomial ($n = 2$, a parabola) to mimic global background bending. If you increase this parameter to a high order (e.g., $n = 6$), the background function becomes highly flexible. It will bend *into* your molecular bands, treating real peaks as background and subtracting your chemical signal out of existence.

3 Working with Your Own Lab Files (.csv)

The system includes a custom parser designed specifically to process raw CSV exports from commercial lab software (such as Bruker OPUS, Agilent MicroLab, or Shimadzu LabSolutions).

3.1 File Formatting Constraints

Before executing your script, open your experimental data inside a text editor or spreadsheet to verify these three critical criteria:

1. **Row 1 (Line 1):** Contains arbitrary instrument information or timestamp text (which python will skip).
2. **Row 2 (Line 2):** Must contain your exact structural header labels. These columns must be typed explicitly as `cm-1` (lowercase, no spaces) for wavenumbers and `%T` for percent transmittance.
3. **Delimiter:** Ensure the file uses standard commas as delimiters.

3.2 The Conversion Calculation

Because spectrometers traditionally collect data in Transmittance mode ($\%T$), while quantitative chemical analysis (Beer-Lambert Law) requires linear scaling, the script converts your raw data into Absorbance values (A) using the exact relationship:

$$A = 2 - \log_{10}(\%T) \tag{1}$$

4 Custom Tuning Guide for Peak Picking

Once your background is clean and flat, the peak finder catalogs your chemical bands. If your automated analysis returns empty arrays or marks noise spikes incorrectly, adjust these two numeric terms in the final function cell:

Table 1: Peak-Finding Tuning Parameters

Python Parameter	Physical Lab Equivalent	Tuning Direction
<code>prominence</code>	Minimum peak height relative to noise floor	Lower it to find trace/weak bands. Raise it if noise spikes get labeled.
<code>distance</code>	Minimum pixel separation between peak maxima	Lower it to separate overlapping split bands. Raise it to ignore shoulder artifacts.

4.1 Quick Parameter Tuning Reference Snippet

To analyze your custom uploaded data file, change the filename string at the bottom of your workspace and adjust your threshold thresholds like this:

```

1 # Step 1: Point to your uploaded CSV file
2 analyzed_data = process_user_micro_csv('my_raw_ftir_export.csv')
3
4 # Step 2: Fine-tune the sensitivity rules for your specific compound
5 # Lower prominence to 0.02 if your fingerprint peaks are weak/subtle
6 identify_and_plot_peaks(analyzed_data, prominence=0.03, distance=15)

```